

RetailSense AI

Plateforme intelligente d'analyse et de prédiction pour le commerce de détail

Diplôme de Technicien en Intelligence Artificielle · Formation de 18 mois

Projet capstone — durée indicative : 4 semaines (≈ 1 mois)

College CDI

Professeur encadrant : Reda Mohammed Chatou

Python

SQLite

Pandas / NumPy

Power BI

Scikit-learn

TensorFlow / Keras

FastAPI

Flutter / Dart

Docker

Document de cadrage à destination des étudiants — de la conception au déploiement.

Comprend : périmètre fonctionnel, déroulé étape par étape, planning, livrables et grille d'évaluation.

1. Présentation générale du projet

RetailSense AI est un projet d'intégration fil rouge conçu pour clôturer la formation de Technicien en Intelligence Artificielle. Il place l'étudiant dans la peau d'un membre de l'équipe Data & IA d'une enseigne fictive de commerce de détail, NovaMarket, qui exploite à la fois des magasins physiques et une boutique en ligne. L'entreprise dispose d'un volume croissant de données (ventes, clients, produits, avis, images de catalogue) mais ne les valorise pas encore. La direction souhaite une plateforme unique capable d'analyser ces données, d'en tirer des prédictions exploitables et de les restituer aux équipes métier via des tableaux de bord et une application mobile.

L'intérêt pédagogique est de relier, dans un seul produit cohérent, l'ensemble des briques vues pendant les 18 mois : programmation Python, bases de données, science des données, machine learning, deep learning, visualisation et développement applicatif multiplateforme, jusqu'au déploiement. Chaque module répond à un besoin métier réaliste plutôt qu'à un exercice isolé, ce qui en fait un projet de portfolio crédible face à un recruteur.

1.1 Problématique métier

- **Comprendre la clientèle** : qui sont les clients, comment les segmenter et lesquels risquent de partir (attrition / churn) ?
- **Anticiper l'activité** : quelle sera la demande des prochaines semaines pour mieux gérer les stocks et les approvisionnements ?
- **Protéger l'entreprise** : comment détecter automatiquement les transactions anormales ou potentiellement frauduleuses ?
- **Augmenter les ventes** : comment recommander à chaque client les produits les plus pertinents et exploiter les avis et les images du catalogue ?
- **Diffuser l'information** : comment mettre tous ces résultats à disposition des équipes (direction, marketing, logistique) de façon claire et accessible sur mobile ?

1.2 Objectifs pédagogiques

- Mener un projet de data science complet, de la collecte des données jusqu'au déploiement en production.
- Mobiliser et articuler tous les chapitres de la formation au sein d'un livrable unique et cohérent.
- Adopter de bonnes pratiques d'ingénierie : structuration de projet, gestion de versions (Git), documentation, reproductibilité et tests.
- Savoir choisir, entraîner, évaluer, comparer et justifier des modèles d'apprentissage automatique et profond.
- Restituer des résultats techniques à un public métier (visualisations, application, présentation orale).
- Produire un projet de portfolio professionnel, défendable en entretien d'embauche.

1.3 Modalités proposées

Paramètre	Recommandation
Durée	≈ 4 semaines (1 mois) à temps plein, ou 6–8 semaines en rythme alterné.
Format	Individuel ou binôme/trinôme. En groupe, attribuer un responsable par module et un intégrateur.
Encadrement	Un point d'avancement hebdomadaire (revue de sprint) + une soutenance finale de 20–30 min.
Données	Jeux de données publics (section 4) ou données synthétiques générées par les étudiants.
Évaluation	Grille détaillée en section 9 (technique, intégration, livrables, communication).

2. Cartographie des compétences mobilisées

Le tableau ci-dessous établit la correspondance entre chaque chapitre de la formation et la partie du projet qui le réinvestit. Il sert de garantie de couverture : à la fin du projet, chaque compétence enseignée a été remise en pratique au moins une fois.

Chapitre de la formation	Où il est réinvesti dans RetailSense AI
1 • Programmation Python	Cœur de tout le projet : scripts de traitement, modules réutilisables, API backend, structuration en packages, classes et fonctions.
2 • Bases de données (SQLite)	Conception du schéma relationnel, création de la base, alimentation (ETL), requêtes d'agrégation servant l'analyse et les modèles.
3 • Applications multiplateformes (Flutter / Dart)	Application mobile/desktop qui consomme l'API : connexion, tableaux de bord, recommandations, prédiction de churn et de demande.
4 • Analyse de données & visualisation (Power BI)	Tableaux de bord décisionnels (ventes, clients, segments, prévisions), KPI et rapports destinés aux équipes métier.
5 • Science des données (NumPy / Pandas)	Nettoyage, fusion, feature engineering, calcul de la matrice RFM, préparation des jeux d'entraînement, manipulation de tableaux numériques.
6 • Machine Learning	Segmentation (K-means, DBSCAN), churn (régression logistique, arbre, Random Forest, AdaBoost, Gradient Boosting), demande/prix (régression linéaire) — avec comparaison de modèles.
7 • Deep Learning	MLP, CNN (images produits), RNN/LSTM (prévision), Transformers (avis), Autoencodeur (anomalies), GAN (données synthétiques), GNN (recommandation) + optimisation et régularisation.

2.1 Concepts transversaux de Deep Learning à démontrer

Ces notions ne constituent pas un module à part : elles doivent apparaître, documentées, dans les modules de deep learning (5.x). L'étudiant explique le choix retenu pour chaque entraînement.

- **Rétropropagation** : expliquer le mécanisme et l'illustrer (au moins une fois) par une mise en œuvre manuelle simple.
- **Métriques** : accuracy, précision, rappel, F1, ROC-AUC pour la classification ; R^2 , MSE, MAE pour la régression ; cross-entropy comme fonction de perte.
- **Optimiseurs** : descente de gradient (SGD), Adam, AdamW, RMSProp — comparer l'effet sur la convergence.
- **Régularisation** : L1 et L2, et justification du paramètre choisi.
- **Stabilisation & généralisation** : Batch Normalization, Dropout, Early Stopping et Callbacks (suivi de la perte de validation, sauvegarde du meilleur modèle, réduction du taux d'apprentissage).

3. Architecture globale de la solution

La plateforme suit une architecture en couches classique d'un produit de données. Les données brutes sont ingérées et centralisées dans une base SQLite, transformées avec Pandas/NumPy, puis exploitées par deux familles de consommateurs : les **modèles d'IA** (ML et DL), exposés via une **API** consommée par l'application **Flutter**, et les **tableaux de bord Power BI** destinés au pilotage. Le tout est conteneurisé pour le déploiement.

3.1 Couches et flux de données

Couche	Rôle	Technologies
1 • Sources	Fichiers CSV/JSON de ventes, clients, produits, avis et images.	CSV, JSON, images
2 • Stockage	Base relationnelle centrale, requêtes et agrégations.	SQLite, SQL
3 • Préparation	Nettoyage, jointures, feature engineering, jeux d'entraînement.	Pandas, NumPy
4 • Intelligence	Entraînement et inférence des modèles ML & DL, persistance des modèles.	Scikit-learn, TensorFlow/Keras
5 • Service	Exposition des prédictions et des données via une API REST.	FastAPI / Flask
6 • Restitution	Application multiplateforme + tableaux de bord décisionnels.	Flutter/Dart, Power BI
7 • Déploiement	Conteneurisation, configuration, hébergement et documentation.	Docker, Git, Cloud/VPS

3.2 Schéma fonctionnel (vue logique)

Sources → Base SQLite → Préparation (Pandas/NumPy) → Modèles ML & DL → API REST → Application Flutter. En parallèle, la base SQLite et les tables agrégées alimentent Power BI pour les tableaux de bord. Les modèles entraînés sont sauvegardés (formats .pkl / .keras) et rechargés par l'API sans réentraînement.

Organisation recommandée du dépôt Git

Dossier	Contenu
data/	Données brutes (raw/) et préparées (processed/) — non versionnées si volumineuses.
db/	Script de création du schéma SQLite et base générée.
notebooks/	Analyse exploratoire et prototypage des modèles.
src/	Modules Python : ingestion, features, modèles ML, modèles DL, utilitaires.
models/	Modèles entraînés sérialisés.
api/	Application FastAPI et schémas de requêtes/réponses.
app_flutter/	Application multiplateforme.
powerbi/	Fichier .pbix et captures des tableaux de bord.
docs/	README, rapport, diagrammes, support de maintenance.

4. Jeux de données recommandés

Pour rester proche de l'industrie tout en restant réalisable, on combine quelques jeux de données publics complémentaires. L'étudiant peut aussi générer des données synthétiques (notamment pour les images ou pour enrichir les transactions). L'important est de couvrir les quatre natures de données : **tabulaire**, **séries temporelles**, **texte** et **image**.

Nature	Exemple de source	Usage dans le projet
Tabulaire (transactions)	Online Retail (UCI), Instacart, ou Brazilian E-Commerce (Olist)	Base SQLite, RFM, segmentation, churn, recommandation.
Séries temporelles	Store Item Demand / Rossmann / agrégation des ventes par date	Prévision de la demande (RNN/LSTM, régression).
Texte (avis clients)	Amazon Reviews, Yelp, ou avis Olist	Analyse de sentiment et classification (Transformers).
Image (produits)	Fashion-MNIST, ou jeu d'images produits e-commerce	Catégorisation automatique du catalogue (CNN).

Recommandation : privilégier un jeu de données e-commerce riche (par exemple Olist, qui contient à la fois commandes, clients, produits, avis et géolocalisation) comme socle, puis le compléter par un jeu d'images et, si besoin, par des données synthétiques. Toujours documenter la source, la licence et la période couverte.

5. Déroulé détaillé, étape par étape

Le projet est découpé en dix phases successives mais reliées entre elles. Chaque phase précise ses objectifs, les étapes concrètes à réaliser, les compétences mobilisées et le livrable attendu. Les étudiants sont invités à committer régulièrement et à documenter leurs choix au fur et à mesure.

Phase 0 — Cadrage et mise en place du projet

Objectif

Poser des fondations propres avant d'écrire du code métier : comprendre le besoin, organiser le travail et préparer l'environnement.

Étapes

- Reformuler le besoin métier et définir 4 à 6 cas d'usage prioritaires (segmentation, churn, demande, anomalies, recommandation, avis).
- Créer le dépôt Git et la structure de dossiers (section 3.2) ; rédiger un README initial.
- Mettre en place l'environnement Python (environnement virtuel, fichier requirements.txt) et figer les versions des bibliothèques.
- Choisir et télécharger les jeux de données (section 4) ; vérifier licences et volumétrie.
- Planifier le travail : backlog des tâches, jalons hebdomadaires, répartition des rôles si projet de groupe.

Livrable

Dépôt initialisé, environnement reproductible, note de cadrage (1–2 pages) avec les cas d'usage et le planning.

Phase 1 — Conception et création de la base de données (SQLite)

Objectif

Centraliser les données dans une base relationnelle bien modélisée, qui servira de source unique de vérité.

Étapes

- Modéliser le schéma : entités clients, produits, commandes, lignes de commande, avis, et table d'historique des ventes agrégées.
- Définir clés primaires, clés étrangères, types et contraintes ; produire un diagramme entité-association (MCD/MLD).
- Créer la base SQLite et les tables via un script SQL versionné (création idempotente).
- Écrire un script Python d'ingestion (ETL) qui charge les fichiers sources, nettoie les types et alimente les tables.
- Écrire des requêtes SQL d'agrégation utiles à la suite : chiffre d'affaires par client, par produit, par mois ; nombre de commandes ; dernier achat.
- Vérifier l'intégrité (clés, doublons, valeurs nulles) et indexer les colonnes fréquemment filtrées.

Compétences mobilisées

Chapitre 2 (SQLite, SQL, modélisation), Chapitre 1 (scripts Python d'ingestion).

Livrable

Base SQLite alimentée, script de création, diagramme du schéma et jeu de requêtes d'agrégation documenté.

Phase 2 — Préparation et ingénierie des données (NumPy / Pandas)

Objectif

Transformer les données brutes en jeux de données propres et en variables explicatives prêtes pour l'analyse et la modélisation.

Étapes

- Charger les données depuis SQLite dans des DataFrames Pandas.
- Nettoyer : gestion des valeurs manquantes, des doublons, des valeurs aberrantes et des types (dates, montants).
- Réaliser les jointures entre commandes, clients, produits et avis pour constituer des tables analytiques.
- Construire la matrice **RFM** (Récence, Fréquence, Montant) par client à l'aide d'opérations vectorisées NumPy/Pandas.
- Effectuer le **feature engineering** : variables temporelles, paniers moyens, taux de retour, ancienneté, encodage des catégories, normalisation/standardisation.
- Définir la variable cible de churn (ex. aucun achat depuis N jours) et préparer les jeux d'entraînement/validation/test.
- Documenter chaque transformation et sauvegarder les jeux préparés (data/processed/).

Compétences mobilisées

Chapitre 5 (NumPy, Pandas), Chapitre 1 (Python), Chapitre 2 (lecture SQL).

Livrable

Notebook d'analyse exploratoire commenté + module reproductible de préparation des données + jeux de données prêts.

Phase 3 — Analyse exploratoire et tableaux de bord (Power BI)

Objectif

Comprendre les données et restituer les indicateurs clés sous forme de tableaux de bord lisibles par des non-techniciens.

Étapes

- Connecter Power BI à la base SQLite ou aux fichiers préparés, et construire le modèle de données (relations entre tables).
- Définir les principaux KPI : chiffre d'affaires, nombre de commandes, panier moyen, taux de churn, top produits, répartition géographique.
- Créer des mesures DAX (CA mensuel, évolution, cumul, taux de croissance).
- Concevoir au moins trois pages de tableau de bord : vue Direction (KPI globaux), vue Clients (segments, churn), vue Produits/Ventes (tendances, prévisions).
- Soigner la mise en forme : filtres interactifs, hiérarchies temporelles, infobulles, cohérence visuelle.
- Exporter un rapport et préparer des captures pour le portfolio.

Compétences mobilisées

Chapitre 4 (Power BI, visualisation, DAX), Chapitre 5 (préparation préalable des données).

Livrable

Fichier .pbix avec au moins trois pages interactives + un rapport de synthèse exporté.

Phase 4 — Machine Learning

Cette phase regroupe trois sous-projets de ML complémentaires. Pour chacun, l'étudiant définit le problème, choisit les variables, entraîne plusieurs modèles, les évalue avec des métriques adaptées et justifie le modèle retenu.

4.1 Segmentation de la clientèle (apprentissage non supervisé)

- Utiliser la matrice RFM (et d'autres variables comportementales) pour regrouper les clients.
- Appliquer **K-means** : déterminer le nombre de clusters (méthode du coude, score de silhouette) et interpréter les segments.
- Appliquer **DBSCAN** : repérer des groupes de densité et isoler les clients atypiques ; comparer avec K-means.
- Nommer les segments d'un point de vue métier (ex. « clients fidèles à forte valeur », « clients dormants ») et les renvoyer dans la base / Power BI.

4.2 Prédiction de l'attrition client — churn (classification supervisée)

- Constituer le jeu étiqueté (client actif / parti) à partir de la phase 2.
- Entraîner et comparer : **régression logistique**, **arbre de décision**, **Random Forest**, **AdaBoost** et **Gradient Boosting**.
- Gérer le déséquilibre des classes si nécessaire (rééchantillonnage, pondération).
- Évaluer avec accuracy, précision, rappel, F1-score, matrice de confusion et courbe ROC-AUC ; analyser l'importance des variables.
- Sélectionner et sauvegarder le meilleur modèle, en justifiant le compromis performance / interprétabilité.

4.3 Prévision du prix ou de la demande (régression)

- Modéliser une cible continue (ex. quantité vendue ou prix) en fonction de variables explicatives, avec une **régression linéaire** comme référence (baseline).
- Comparer à un modèle d'ensemble (Random Forest / Gradient Boosting) sur la même tâche.
- Évaluer avec R^2 , MSE et MAE ; analyser les résidus.

Compétences mobilisées

Chapitre 6 (tous les algorithmes ML, théorie et évaluation), Chapitre 5 (préparation), Chapitre 1 (Python).

Livrable

Modèles entraînés et sérialisés, tableau comparatif des performances, et note d'interprétation des segments et des prédictions.

Phase 5 — Deep Learning

Sept modules de deep learning, chacun rattaché à un besoin métier. Tous doivent illustrer explicitement les concepts transversaux de la section 2.1 (perte, optimiseur, régularisation, Dropout, Batch Normalization, Early Stopping, Callbacks) et présenter les courbes d'apprentissage. En projet de groupe, ces modules se répartissent bien entre les membres.

5.1 MLP — prédiction tabulaire

Réimplémenter la tâche de churn (ou de demande) avec un perceptron multicouche, et comparer ses performances à celles des modèles ML de la phase 4. Occasion de mettre en œuvre manuellement une rétropropagation simple à des fins pédagogiques.

5.2 CNN — catégorisation des images produits

Entraîner un réseau convolutif pour classer automatiquement les images du catalogue par catégorie (et/ou évaluer la qualité d'image). Aborder l'augmentation de données et, en bonus, le transfert d'apprentissage à partir d'un modèle pré-entraîné.

5.3 RNN / LSTM — prévision de la demande

Modéliser les ventes agrégées comme une série temporelle et prévoir les semaines suivantes avec un RNN puis un LSTM ; comparer au modèle de régression de la phase 4.3. Alimenter la planification des stocks.

5.4 Transformers — analyse des avis clients

Classer le sentiment (positif/négatif/neutre) ou la note des avis textuels à l'aide d'un Transformer (modèle pré-entraîné affiné). Les résultats enrichissent la connaissance client et les tableaux de bord.

5.5 Autoencodeur — détection d'anomalies / fraude

Entraîner un autoencodeur sur les transactions « normales » et utiliser l'erreur de reconstruction pour signaler les transactions atypiques, potentiellement frauduleuses.

5.6 GAN — génération de données synthétiques

Entraîner un GAN pour générer des données synthétiques (images produits supplémentaires, ou échantillons tabulaires pour équilibrer une classe rare), puis vérifier l'apport sur une tâche aval.

5.7 GNN — système de recommandation

Représenter les interactions clients-produits sous forme de graphe et entraîner un réseau de neurones sur graphe pour recommander des produits. Comparer, en option, à une approche de filtrage collaboratif classique. Les recommandations sont exposées dans l'application Flutter.

Compétences mobilisées

Chapitre 7 (toutes les architectures, optimisation, régularisation, métriques), Chapitre 5 (données), Chapitre 1 (Python).

Livable

Un notebook ou module par architecture, modèles sauvegardés, courbes d'apprentissage et synthèse comparant DL et ML lorsque pertinent.

Phase 6 — Backend et API d'inférence

Objectif

Exposer les modèles et les données via une API REST stable, afin que l'application et d'autres clients puissent les consommer.

Étapes

- Concevoir les points d'accès (endpoints) : segmentation d'un client, prédiction de churn, prévision de demande, recommandations, score d'anomalie, sentiment d'un avis.
- Charger les modèles sérialisés au démarrage et écrire la logique d'inférence (prétraitement identique à l'entraînement).
- Valider les entrées/sorties (schémas), gérer les erreurs et documenter automatiquement l'API (ex. Swagger/OpenAPI).
- Ajouter quelques tests d'intégration sur les endpoints clés.

Compétences mobilisées

Chapitre 1 (Python, structuration, API), Chapitres 6 et 7 (chargement et inférence des modèles).

Livrable

API REST documentée et testée, exposant l'ensemble des services d'IA.

Phase 7 — Application multiplateforme (Flutter / Dart)

Objectif

Offrir aux équipes métier une application unique (mobile/desktop) pour consulter les indicateurs et interroger les modèles.

Étapes

- Concevoir l'expérience : écran de connexion, tableau de bord synthétique, fiche client (segment + risque de churn), prévision de la demande, recommandations produits.
- Mettre en place l'architecture de l'app (gestion d'état, navigation) et consommer l'API de la phase 6 (appels HTTP, sérialisation JSON).
- Afficher les résultats sous forme de graphiques et de listes claires ; gérer les états de chargement et d'erreur.
- Soigner l'ergonomie et la cohérence visuelle ; tester sur au moins deux plateformes (Android + desktop ou web).

Compétences mobilisées

Chapitre 3 (Flutter, Dart, multiplateforme), Chapitre 1 (logique applicative).

Livrable

Application Flutter fonctionnelle connectée à l'API, avec au moins quatre écrans utiles et une courte démonstration vidéo.

Phase 8 — Déploiement et industrialisation légère

Objectif

Rendre la solution exécutable en dehors du poste de développement et démontrer une démarche de mise en production.

Étapes

- Conteneuriser l'API (et la base) avec Docker ; rédiger un Dockerfile et, si pertinent, un docker-compose.
- Externaliser la configuration (variables d'environnement) et gérer les chemins des modèles.
- Déployer l'API sur un hébergement accessible (VPS, plateforme cloud gratuite ou conteneur géré) et connecter l'application.
- Mettre en place un suivi minimal : journalisation des requêtes, gestion des versions de modèles, et procédure de réentraînement documentée (notions de MLOps).
- Vérifier la reproductibilité de bout en bout à partir du README.

Compétences mobilisées

Chapitre 1 (Python, packaging), bonnes pratiques d'ingénierie et de déploiement transversales.

Livrable

API déployée et accessible, image Docker, et documentation de déploiement reproductible.

Phase 9 — Documentation, tests et préparation du portfolio

Objectif

Transformer le travail réalisé en un livrable professionnel, compréhensible et valorisable en entretien.

Étapes

- Rédiger un README complet : contexte, architecture, instructions d'installation et d'exécution, captures d'écran.
- Rédiger un rapport technique (15–25 pages) décrivant les choix, les résultats chiffrés et les limites.
- Nettoyer le code (lisibilité, commentaires, suppression du code mort) et compléter les tests.
- Préparer un support de soutenance orienté résultats et démonstration (10–15 diapositives).
- Publier le projet sur un dépôt public soigné et, idéalement, mettre en ligne une démonstration.

Livrable

Dépôt documenté, rapport technique, support de présentation et démonstration prêts pour le portfolio.

6. Planning indicatif sur 4 semaines

Répartition proposée pour un mois à temps plein. En projet de groupe, plusieurs modules de la semaine 2 et 3 sont menés en parallèle.

Semaine	Focus principal	Phases couvertes	Jalon de fin de semaine
Semaine 1	Données & restitution analytique	Phases 0 à 3	Base SQLite alimentée + tableaux de bord Power BI.
Semaine 2	Machine Learning	Phase 4 (+ début 5)	Segmentation, churn et prévision opérationnels.
Semaine 3	Deep Learning	Phase 5	Modules DL entraînés et évalués.
Semaine 4	Intégration, déploiement & portfolio	Phases 6 à 9	API + app Flutter déployées, documentation et maintenance.

Astuce : la semaine 4 est la plus risquée car elle concentre l'intégration. Encourager les étudiants à figer une version « bout en bout » minimale dès la fin de la semaine 2 (base → un modèle → API simple → un écran), puis à enrichir, plutôt que d'intégrer seulement à la fin.

7. Livrables attendus

- Dépôt Git public, structuré et documenté (README, instructions reproductibles).
- Base de données SQLite + script de création et diagramme du schéma.
- Notebooks d'analyse exploratoire et de prototypage.
- Modules Python de préparation des données et d'entraînement des modèles.
- Modèles ML et DL entraînés et sérialisés.
- Fichier Power BI (.pbix) avec au moins trois pages interactives.
- API REST documentée et testée.
- Application Flutter multiplateforme connectée à l'API.
- Configuration de déploiement (Docker) et, idéalement, démonstration en ligne.
- Rapport technique (15–25 pages) et support de maintenance.
- Courte vidéo de démonstration du produit.

8. Grille d'évaluation proposée (100 points)

Grille indicative à adapter. Elle valorise autant la qualité technique que l'intégration, la restitution et la communication — compétences attendues en entreprise.

Critère	Ce qui est évalué	Points
Données & base	Modélisation SQLite, qualité de l'ETL, nettoyage et feature engineering (Pandas/NumPy).	15
Visualisation	Pertinence et lisibilité des tableaux de bord Power BI, KPI et mesures DAX.	10
Machine Learning	Pertinence des modèles, comparaison, métriques adaptées, interprétation.	15
Deep Learning	Architectures, mise en œuvre des concepts (optimisation, régularisation, callbacks), résultats.	20
Intégration & API	API fonctionnelle, application Flutter connectée, cohérence bout en bout.	15
Déploiement & ingénierie	Conteneurisation, reproductibilité, gestion de versions, tests.	10
Documentation	README, rapport technique, clarté du dépôt.	8
Soutenance	Démonstration, justification des choix, qualité de la présentation.	7
Total		100

9. En faire un vrai projet de portfolio

Au-delà de la note, l'objectif est que ce projet aide les étudiants à décrocher un emploi. Quelques recommandations pour le rendre convaincant face à un recruteur :

- **Raconter une histoire** : partir d'un problème métier clair et montrer l'impact, pas seulement les modèles.
- **Montrer les résultats** : chiffrer les performances et illustrer par des captures, des graphiques et une démonstration.
- **Soigner le README** : c'est la première chose que regarde un recruteur ; il doit permettre de lancer le projet en quelques minutes.
- **Mettre en avant l'étendue** : un projet qui couvre données, ML, DL, API, application et déploiement démontre une polyvalence rare chez un débutant.
- **Assumer les limites** : une section « limites et améliorations futures » montre de la maturité et de l'esprit critique.
- **Préparer le pitch** : savoir présenter le projet en 2 minutes, puis répondre à « pourquoi ce modèle ? » et « comment passer à l'échelle ? ».

9.1 Extensions et bonus possibles

- Transfert d'apprentissage pour le CNN et affinage avancé du Transformer.
- Suivi des expériences et des versions de modèles (notions de MLOps).
- Mise en place d'une CI simple (tests automatiques à chaque push).
- Authentification et gestion de rôles dans l'application.
- Tableau de bord temps réel des scores d'anomalies.

9.2 Pile technologique récapitulative

Python · SQLite · Pandas · NumPy · Scikit-learn · TensorFlow/Keras · Power BI · FastAPI (ou Flask) · Flutter/Dart · Docker · Git. Cette combinaison correspond à un profil de jeune praticien en IA directement

opérationnel en entreprise.

Bon projet à vos étudiants : un produit de bout en bout vaut bien plus, dans un portfolio, qu'une collection d'exercices isolés.